

Протокол интеграции постамата

MVP: Android-приложение, Modbus RS-485 и WebSocket backend

Цель MVP: бронирование -> команда открытия -> постабат -> подтверждение открытия/закрытия -> приложение.

Параметр	Значение
Постабат	map-7
Физические ячейки стенда	D1, D2, D3, D4
Транспорт	WebSocket /v1/device/socket
Локальный контроллер	Modbus RTU через USB-RS-485
Статус документа	Готово для интеграции backend

Важно: Android не подключается к MongoDB напрямую. Рабочая схема: Android и Modbus общаются с WebSocket backend, а backend работает с MongoDB/Redis.

Версия документа: 1.0 | 19.09.2026

1. Соответствие ячеек и Modbus

В протоколе backend номер ячейки передается как двухзначный code. В интерфейсе приложения тот же номер показывается в коротком виде D1, D2, D3, D4.

Интерфейс	Вход кнопки	Выход LED	cellCode в JSON	Смысл
D1	X1	Y1	01	open / closed
D2	X2	Y2	02	open / closed
D3	X3	Y3	03	open / closed
D4	X4	Y4	04	open / closed

2. Что уже работает на нашей стороне

- Android-приложение подключается к Modbus RTU через USB-RS-485.
- Команда backend open_cell включает нужный Y-выход.
- LED включен - ячейка считается открытой: D1 open.
- Нажатие X1-X4 выключает соответствующий LED - ячейка считается закрытой: D1 closed.
- WebSocket подключение к тестовому backend работает.
- Сообщения hello, welcome и heartbeat проходят.
- Подтверждения ask и события door_opened / door_closed отправляются.
- В Android есть отдельный журнал событий, сохранение и очистка журнала.

Ограничение MVP: настоящего замка и датчика двери пока нет. LED и кнопка имитируют физические состояния. Поэтому lock = unknown, а занятость ячейки устройство не определяет.

3. Обмен сообщениями

Все сообщения - JSON с полем type. Примеры ниже приведены в формате JSONC для пояснений. Комментарии после // не отправляются по сети.

Android -> backend: hello после подключения

HELLO

```
{
  "type": "hello",
  "postamatId": "map-7",
  "appVersion": "0.1.0",
  "keyVersion": 0,
  "capabilities": ["locks", "buttons", "indicators", "demo_buttons_as_doors"],
  "cells": [
    {"code": "01", "door": "closed", "lock": "unknown"},
    {"code": "02", "door": "closed", "lock": "unknown"},
    {"code": "03", "door": "closed", "lock": "unknown"},
    {"code": "04", "door": "closed", "lock": "unknown"}
  ],
  "net": {"kind": "wifi"}
}
```

Backend -> Android: welcome после hello

WELCOME

```
{
  "type": "welcome",
  "postamatId": "map-7",
  "postamatName": "Постамат map-7",
  "cells": [
    {"code": "01", "door": "closed", "lock": "unknown"},
    {"code": "02", "door": "closed", "lock": "unknown"},
    {"code": "03", "door": "closed", "lock": "unknown"},
    {"code": "04", "door": "closed", "lock": "unknown"}
  ],
  "config": {"heartbeatSec": 15, "pingSec": 20, "pongWaitSec": 45}
}
```

Для физического стенда backend должен использовать только cellCode 01-04. Ячейки 05-08 пока не подключены.

4. Открытие ячейки

Команда приходит от backend после успешной проверки бронирования и оплаты.

BACKEND -> ANDROID: COMMAND OPEN_CELL

```
{
  "type": "command",
  "id": "command-123",
  "kind": "open_cell",
  "payload": {"cellCode": "01"},
  "expiresAt": 1800000000
}
```

Android проверяет срок действия, включает Y1 и отправляет подтверждение:

ANDROID -> BACKEND: ACK

```
{
  "type": "ack",
  "commandId": "command-123",
  "ok": true,
  "result": {"kind": "door_open_ack", "cellCode": "01"}
}
```

ANDROID -> BACKEND: EVENT

```
{
  "type": "event",
  "kind": "door_opened",
  "cellCode": "01",
  "detail": {"source": "demo_relay", "simulated": true}
}
```

Только успешный ack с result.kind = door_open_ack является основанием начать аренду и оплату.

Если ячейка неизвестна, команда просрочена или Modbus недоступен, Android отправляет ack с ok = false. Backend не должен начинать аренду.

5. Заккрытие ячейки

Пользователь нажимает физическую кнопку. Android выключает соответствующий LED и сообщает backend, что дверь закрыта.

ANDROID -> BACKEND: DOOR CLOSED

```
{
  "type": "event",
  "kind": "door_closed",
  "cellCode": "01",
  "detail": {"source": "button", "simulated": true}
}
```

Backend может дополнительно запросить подтверждение:

BACKEND -> ANDROID: CONFIRM CLOSED

```
{
  "type": "command",
  "id": "command-124",
  "kind": "confirm_closed",
  "payload": {"cellCode": "01"}
}
```

ANDROID -> BACKEND: CLOSE ACK

```
{
  "type": "ack",
  "commandId": "command-124",
  "ok": true,
  "result": {"kind": "door_close_ack", "cellCode": "01"}
}
```

6. Heartbeat и потеря связи

Android отправляет heartbeat каждые 15 секунд. Backend обновляет last_seen_at и статус online.

ANDROID -> BACKEND: HEARTBEAT

```
{
  "type": "heartbeat",
  "cells": [
    {"code": "01", "door": "open", "lock": "unknown"},
    {"code": "02", "door": "closed", "lock": "unknown"},
    {"code": "03", "door": "closed", "lock": "unknown"},
    {"code": "04", "door": "closed", "lock": "unknown"}
  ],
  "net": {"kind": "wifi"},
  "problems": []
}
```

- Нет heartbeat около 45 секунд - постапат offline.
- Команда с истекшим expiresAt не должна выполняться после восстановления связи.
- Повторная команда с тем же id не должна открывать дверь повторно.
- Серверный ping должен получать обычный WebSocket pong.

7. Что должен завершить backend

Задача	Ожидаемый результат
Бронирование и оплата	После проверки оплаты отправить open_cell для нужного cellCode.
Начало аренды	Начинать только после door_open_ack с ok=true.
Закрытие аренды	Завершать после door_closed и подтверждения закрытия.
Статусы для приложения	Показывать D1 open, D1 closed и аналогично для D2-D4.
Online/offline	Контролировать heartbeat и last_seen_at.
Безопасность команд	Проверять expiresAt, commandId и повторную доставку.
Ошибки	Обработать по ack, modbus_offline, door_not_closed и потерю связи.
Ячейки стенда	Для map-7 использовать только 01-04.

Итог: Android, Modbus, тестовый WebSocket и обратная связь уже работают. Backend нужно связать с бронированием, оплатой, арендой, статусами пользователя и offline-сценариями.

Короткие состояния для интерфейса: D1 open / D1 closed / D2 open / D2 closed / D3 open / D3 closed / D4 open / D4 closed.